BiyaheMate: A Real-Time Public Transport Information System for Filipino Commuters

Prepared by: Comeros, John Marc
Date: 10/14/26
Version: 1.0

---

1. PROBLEM

1.1 Background

Daily commuting in the Philippines, especially in Metro Manila and other urban areas, is a major source of stress, lost time, and financial burden. Millions of Filipinos rely on public utility vehicles (PUVs) such as jeepneys, buses, UV Express, and trains. However, the lack of real-time information makes it difficult to plan trips and predict arrival times.

1.2 Key Issues

- Massive time loss: Commuters lose an average of 127 hours per year to traffic congestion in Metro Manila alone. A single trip can take over an hour for short distances.
- Unpredictability and long waits: There is no reliable information on when the next jeepney or bus will arrive. Commuters often wait 30 minutes to an hour, described locally as “hula-hula lang” (just guessing).
- Financial burden: Public transport costs average ₱108 per day, which is about 16% of the daily minimum wage in Metro Manila. This is a heavy burden for low-income workers.
- Inadequate information: Existing apps like Google Maps and Waze are designed for private vehicles. Local alternatives often have outdated or incomplete data and do not capture the complex, multimodal nature of Philippine public transport.
- Lack of real-time updates: Traffic accidents, road closures, and full jeepneys are not reported in real time, leaving commuters stranded or delayed.

1.3 Target Users

- Students
- Office workers
- Daily commuters
- Tourists and occasional riders

- Transport operators and cooperatives (future)

1.4 Existing Gaps

- No centralized, real-time platform for PUV tracking.
- No crowdsourced reporting for traffic and vehicle conditions.
- No multimodal route planning that combines jeepney, bus, train, and walking.
- No direct communication channel between operators and commuters.

---

2. SOLUTION

2.1 Proposed System

BiyaheMate is a web-based, mobile-first Progressive Web App (PWA) that provides real-time public transport information. It uses crowdsourced data and a Backend-as-a-Service (BaaS) to deliver live vehicle locations, accurate arrival estimates, route planning, and community alerts.

2.2 Core Features

- Real-time vehicle tracking: Display live locations of jeepneys, buses, and other PUVs on a map. Initial data comes from crowdsourced sightings; later, GPS integration with operators.
- Accurate arrival estimates (ETA): Calculate dynamic ETAs based on real-time location and traffic conditions.
- Multimodal route planning: Suggest routes combining walking, jeepney, bus, and train.
- Crowdsourced reports and alerts: Users can report heavy traffic, road closures, full vehicles, or accidents. Alerts are pushed to other users.
- Operator dashboard (future): A portal for transport cooperatives to manage routes, schedules, and announcements.
- User accounts and favorites: Save frequent routes and stops for quick access.

2.3 Value Proposition

- Save time by reducing waiting and guessing.
- Reduce stress and uncertainty in daily travel.

- Save money by choosing more efficient routes.
- Empower commuters with real-time, community-driven information.

---

3. PLANNING

3.1 Goals and Objectives

- Build a functional MVP within 8–12 weeks as a solo frontend developer.
- Use Supabase as the backend-as-a-service to avoid custom server code.
- Deploy on Vercel with separate Supabase and Vercel accounts.
- Validate the concept with real users before adding complex GPS hardware.

3.2 Scope

MVP (Phase 1–3):

- User authentication (email/password).
- Interactive map with vehicle markers (crowdsourced).
- Search destination and view route suggestions.
- Submit and view crowdsourced reports.
- Real-time updates using Supabase Realtime.
- Basic ETA calculation.
- Mobile-responsive PWA.

Future (Phase 4+):

- Operator dashboard.
- Integration with GPS devices on PUVs.
- Push notifications for alerts.
- Fare calculator with updated fare matrices.
- Offline mode for poor connectivity.

3.3 Development Phases

Phase 1: Foundation (Weeks 1–2)

- Set up Next.js, Tailwind CSS, Leaflet.
- Create static UI screens with mock data.
- Set up GitHub repository.

Phase 2: Supabase Integration (Weeks 3–5)

- Create Supabase project.
- Set up authentication, database tables, and basic queries.
- Replace mock data with real data from Supabase.
- Implement user login and profile.

Phase 3: Real-Time and Crowdsourcing (Weeks 6–8)

- Enable Supabase Realtime for vehicle locations and reports.
- Build report submission form with geolocation.
- Implement live map updates.
- Add basic ETA logic.

Phase 4: PWA and Polish (Weeks 9–12)

- Convert to PWA with offline caching.
- Add alerts and favorites.
- User testing and bug fixes.
- Deploy to production on Vercel.

3.4 Tech Stack

- Frontend: Next.js (React), Tailwind CSS, Leaflet, OpenStreetMap tiles.
- Backend: Supabase (Auth, PostgreSQL, Realtime, Storage).
- Hosting: Vercel.
- Version Control: GitHub.
- Design: Figma (optional).

3.5 Resources

- Solo frontend developer.

- Free tiers of Supabase and Vercel.
- OpenStreetMap for map tiles (free).
- No custom server required.

3.6 Risks and Mitigations

- Risk: Inaccurate crowdsourced data.
  Mitigation: Allow users to upvote/downvote reports; moderator review.
- Risk: Low user adoption.
  Mitigation: Partner with local commuter groups; simple, fast UI.
- Risk: Poor internet connectivity.
  Mitigation: PWA offline caching; lightweight map tiles.
- Risk: Real GPS tracking is hard.
  Mitigation: Start with crowdsourced sightings; add operator partnerships later.

---

4. SYSTEM DESIGN

4.1 User Roles

- Commuter (Guest/User): View map, search routes, submit reports, receive alerts.
- Moderator/Admin: Manage reports, users, and alerts.
- Operator (Future): Manage routes and vehicle data.

4.2 Functional Requirements

- Map view with live vehicle markers.
- Search destination and display route suggestions.
- Real-time ETA for selected routes.
- Submit crowdsourced report (type, location, description).
- View and filter reports on the map.
- Receive alerts for disruptions.
- Save favorite routes and stops.
- User registration and login.

- Admin moderation of reports.

4.3 Non-Functional Requirements

- Responsive design for mobile and desktop.
- PWA installable on phones.
- Page load under 3 seconds on 4G.
- Secure authentication and data access.
- Scalable to thousands of users using Supabase.
- Accessible (WCAG 2.1 AA basics).

4.4 Database Schema (Supabase / PostgreSQL)

Table: users

- id (uuid, primary key, references auth.users)
- email (text)
- role (text: commuter, admin, operator)
- created_at (timestamp)

Table: routes

- id (uuid, primary key)
- name (text)
- origin (text)
- destination (text)
- fare (numeric)
- geometry (jsonb) – optional path coordinates

Table: stops

- id (uuid, primary key)
- route_id (uuid, foreign key)
- name (text)
- latitude (numeric)

- longitude (numeric)

Table: vehicles

- id (uuid, primary key)
- route_id (uuid, foreign key)
- latitude (numeric)
- longitude (numeric)
- last_updated (timestamp)
- status (text: active, full, inactive)

Table: reports

- id (uuid, primary key)
- user_id (uuid, foreign key)
- type (text: traffic, full, accident, closure)
- latitude (numeric)
- longitude (numeric)
- description (text)
- created_at (timestamp)
- status (text: pending, approved, rejected)

Table: alerts

- id (uuid, primary key)
- title (text)
- message (text)
- severity (text: info, warning, critical)
- created_at (timestamp)
- expires_at (timestamp)

Table: favorites

- id (uuid, primary key)

- user_id (uuid, foreign key)
- route_id (uuid, foreign key, nullable)
- stop_id (uuid, foreign key, nullable)
- created_at (timestamp)

4.5 UI Screens

- Home / Map: Full-screen map with vehicle markers and report pins.
- Search: Input destination, list of suggested routes.
- Route Details: Step-by-step directions, ETA, fare.
- Report: Form to submit a crowdsourced report.
- Alerts: List of active alerts.
- Profile: User info, favorites, logout.
- Admin: Moderation dashboard (future).

---

5. ARCHITECTURE

5.1 High-Level Architecture

The system uses a serverless, BaaS-driven architecture. There is no custom backend server. The frontend communicates directly with Supabase for authentication, database, and realtime updates. Map tiles are fetched from OpenStreetMap.

Components:

- Client: Next.js PWA hosted on Vercel.
- Backend-as-a-Service: Supabase (Auth, PostgreSQL, Realtime, Storage).
- Map: Leaflet library with OpenStreetMap tiles.
- Hosting: Vercel for the frontend.
- Version Control: GitHub.

5.2 Architecture Diagram (Text Representation)

[User Device (Browser/PWA)]
|

| HTTPS
v
[Vercel (Next.js Frontend)]
|
| Supabase JS Client
v
[Supabase]

- Auth (JWT)
- PostgreSQL Database
- Realtime Subscriptions
- Storage (optional)
  |
  | Realtime Broadcast
  v
  [Other Users' Devices]

[Leaflet + OpenStreetMap Tiles] <-- Direct tile requests from client

5.3 Data Flow

1. User opens the app. Next.js loads and authenticates via Supabase.
2. The app fetches active vehicles, routes, and reports from Supabase.
3. Leaflet renders the map and markers.
4. When a user submits a report, the app inserts a row into the reports table.
5. Supabase Realtime broadcasts the new report to all subscribed clients.
6. Other users see the new report on their map in real time.
7. For vehicle tracking, a user can share their location while on a PUV. The app updates the vehicles table, and other users see the marker move.

5.4 Environment Variables
Set these in Vercel Project Settings > Environment Variables:

- NEXT_PUBLIC_SUPABASE_URL
- NEXT_PUBLIC_SUPABASE_ANON_KEY
- NEXT_PUBLIC_MAPBOX_TOKEN (optional if using Mapbox)

- NEXT_PUBLIC_MAP_TILE_URL (optional, default OpenStreetMap)

5.5 Deployment

- Code pushed to GitHub.
- Vercel automatically deploys on push.
- Supabase project can be on a separate account. Manually copy the Project URL and anon key from Supabase Dashboard > Project Settings > API into Vercel environment variables.
- No account linking required. This is the simplest and most reliable method.

---

6. FLOWS

6.1 User Opens App and Views Map

1. User navigates to the web app.
2. Next.js loads, checks for Supabase session.
3. If not logged in, user can still view the map as a guest.
4. App queries vehicles and reports tables.
5. Leaflet displays map with markers.
6. Supabase Realtime subscription is established for live updates.

6.2 User Searches for a Route

1. User taps the search bar and enters a destination.
2. App queries routes and stops tables for matching destinations.
3. App calculates possible routes (simple matching or external API).
4. User selects a route.
5. App displays step-by-step directions, estimated fare, and ETA.
6. User can save the route to favorites (requires login).

6.3 User Submits a Crowdsourced Report

1. User taps the “Report” button.
2. App requests geolocation permission.

3. User selects report type (traffic, full, accident, closure).
4. User adds optional description.
5. App inserts a new row into the reports table with user_id, type, coordinates, and timestamp.
6. Supabase Realtime broadcasts the new report.
7. All connected users see the new report pin on the map.

6.4 Real-Time Vehicle Tracking via Crowdsourcing

1. User is riding a jeepney and taps “I’m on this vehicle.”
2. App starts sending the user’s geolocation to the vehicles table every 10–15 seconds.
3. The vehicle marker updates on all users’ maps in real time.
4. When the user taps “I’m off,” the vehicle status is set to inactive or removed after a timeout.
5. Later, operators can provide GPS data directly.

6.5 User Receives Alerts

1. Admin creates an alert in the alerts table.
2. Supabase Realtime broadcasts the alert.
3. App displays a banner or notification to all connected users.
4. Alert remains visible until expires_at.

6.6 Authentication Flow

1. User taps “Sign Up” or “Log In.”
2. App calls Supabase Auth with email and password.
3. Supabase returns a JWT and user session.
4. App stores session and redirects to profile or map.
5. User can now submit reports, save favorites, and access personalized features.

6.7 Admin Moderation Flow

1. Admin logs in with an admin role.
2. Admin dashboard lists pending reports.

3. Admin can approve, reject, or delete reports.
4. Approved reports remain visible; rejected reports are hidden.
5. Admin can also create and manage alerts.

6.8 Deployment Flow

1. Developer writes code locally.
2. Code is pushed to GitHub repository.
3. Vercel detects the push and builds the Next.js app.
4. Vercel injects environment variables (Supabase URL and anon key).
5. App is deployed to a production URL.
6. Supabase remains on its own account; no direct linking needed.

---

7. APPENDICES

7.1 Sample SQL for Supabase Tables
You can run this in the Supabase SQL Editor.

```
CREATE TABLE users (
id uuid REFERENCES auth.users PRIMARY KEY,
email text,
role text DEFAULT 'commuter',
created_at timestamp DEFAULT now()
);

CREATE TABLE routes (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
name text,
origin text,
destination text,
fare numeric,
geometry jsonb
);

CREATE TABLE stops (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
route_id uuid REFERENCES routes(id),
```

```
name text,
latitude numeric,
longitude numeric
);

CREATE TABLE vehicles (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
route_id uuid REFERENCES routes(id),
latitude numeric,
longitude numeric,
last_updated timestamp DEFAULT now(),
status text DEFAULT 'active'
);

CREATE TABLE reports (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
user_id uuid REFERENCES users(id),
type text,
latitude numeric,
longitude numeric,
description text,
created_at timestamp DEFAULT now(),
status text DEFAULT 'pending'
);

CREATE TABLE alerts (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
title text,
message text,
severity text,
created_at timestamp DEFAULT now(),
expires_at timestamp
);

CREATE TABLE favorites (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
user_id uuid REFERENCES users(id),
route_id uuid REFERENCES routes(id),
stop_id uuid REFERENCES stops(id),
```

```
created_at timestamp DEFAULT now()
);
```

7.2 Sample Supabase Client Code (Next.js)

```
import { createClient } from '@supabase/supabase-js'

const supabase = createClient(
process.env.NEXT_PUBLIC_SUPABASE_URL,
process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY
)

// Fetch reports
const { data: reports, error } = await supabase
.from('reports')
.select('*')
.order('created_at', { ascending: false })

// Realtime subscription
const subscription = supabase
.channel('public:reports')
.on('postgres_changes', { event: 'INSERT', schema: 'public', table: 'reports' }, payload => {
console.log('New report:', payload.new)
})
.subscribe()
```

7.3 Environment Variables Checklist

- NEXT_PUBLIC_SUPABASE_URL = https://your-project.supabase.co
- NEXT_PUBLIC_SUPABASE_ANON_KEY = your-anon-key
- NEXT_PUBLIC_MAP_TILE_URL = https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png (optional)